I6 · Écrire ses propres fonctions

~3 h

> À la fin de cette séance vous saurez
>
> - écrire une **fonction** avec des arguments, des valeurs par défaut et un résultat renvoyé ;
> - **vérifier** les arguments et arrêter avec un message clair quand ils sont faux ;
> - renvoyer plusieurs résultats dans une **liste** ;
> - appliquer votre fonction à chaque groupe avec `sapply()`, `tapply()` ou `group_by()` ;
> - ranger vos fonctions dans un fichier séparé et les charger avec `source()`.

⇩ checkpoint: I3_seaweed_analysis.csv

Pourquoi écrire des fonctions ?

Depuis la séance I1 vous calculez le taux de croissance avec `100 * log(final / initial) / days`. Vous en aurez encore besoin pour les holothuries du projet final, pour chaque village, pour un rapport… Une **fonction** donne un nom à ce calcul : vous l'écrivez et la vérifiez **une fois**, et chaque utilisation est courte et identique. Règle pratique : **si vous avez copié le même code trois fois, écrivez une fonction.**

Anatomie d'une fonction

```
sgr <- function(w_initial, w_final, days) {     # name <- function(arguments)
  100 * log(w_final / w_initial) / days         # the LAST value is returned
}
sgr(10, 25, 45)                                  # one line
```

```
#> [1] 2.036202
```

```
sgr(w_initial = 10, w_final = c(25, 30), days = 45)   # works on vectors too
```

```
#> [1] 2.036202 2.441361
```

- Les **arguments** sont des noms locaux : dans la fonction, `w_initial` vaut ce que vous lui passez. Les objets créés dans la fonction disparaissent à la fin — ils n'écrasent pas vos objets.
- Le résultat est la **dernière valeur** calculée. `return(x)` le rend explicite (utile pour sortir plus tôt).
- Comme le calcul est vectorisé, la fonction marche sur des colonnes entières :

```
seaweed$sgr2 <- sgr(seaweed$initial_weight_kg, seaweed$final_weight_kg, seaweed$da
ys)
all.equal(seaweed$sgr, seaweed$sgr2)        # same as Session I1
```

```
#> [1] TRUE
```

Valeurs par défaut

Donnez une valeur par défaut à un argument avec `=` : l'utilisateur peut l'omettre. Ici, une fonction de résumé avec `digits = 2` et `na.rm = TRUE` par défaut :

```
describe <- function(x, digits = 2, na.rm = TRUE) {
  n  <- sum(!is.na(x))
  m  <- mean(x, na.rm = na.rm)
  s  <- sd(x, na.rm = na.rm)
  round(c(n = n, mean = m, sd = s, se = s / sqrt(n)), digits)
}
describe(seaweed$sgr)
```

```
#>       n    mean      sd      se
#> 2317.00    3.44    0.67    0.01
```

```
describe(seaweed$sgr, digits = 4)
```

```
#>         n      mean        sd        se
#> 2317.0000    3.4387    0.6659    0.0138
```

Vérifier les arguments

Une bonne fonction **refuse les mauvaises entrées avec un message clair**, au lieu de renvoyer un résultat faux en silence. `stop()` arrête avec une erreur ; `warning()` continue avec un avertissement.

```
sgr <- function(w_initial, w_final, days) {
  if (!is.numeric(w_initial) || !is.numeric(w_final)) stop("weights must be number
s")
  if (any(days <= 0, na.rm = TRUE)) stop("days must be positive")
  if (any(w_final > 100 * w_initial, na.rm = TRUE))
    warning("some final weights are > 100 x the initial weight: grams instead of k
g?")
  100 * log(w_final / w_initial) / days
}
sgr("10", 25, 45)
```

```
#> Error in sgr("10", 25, 45): weights must be numbers
```

```
sgr(10, 2500, 45)
```

```
#> Warning in sgr(10, 2500, 45): some final weights are > 100 x the initial
#> weight: grams instead of kg?
```

```
#> [1] 12.26991
```

`||` et `&&` sont les versions « valeur unique » de `|` et `&`, utilisées dans `if()`.

Renvoyer plusieurs résultats : les listes

Une fonction renvoie **un seul** objet. Pour renvoyer plusieurs résultats de types différents, mettez-les dans une **liste** (un objet qui peut tout contenir) et accédez aux éléments avec `$`.

```
farm_report <- function(d) {
  list(n_lines     = nrow(d),
       loss_pct    = round(100 * mean(d$status != "harvested"), 1),
       harvest_t   = round(sum(d$final_weight_kg, na.rm = TRUE) / 1000, 2),
       best_farmer = names(which.max(tapply(d$sgr, d$farmer_id, mean, na.rm = TRU
E))))
}
r <- farm_report(seaweed[seaweed$village == "Lovobe", ])
r
```

```
#> $n_lines
#> [1] 220
#>
#> $loss_pct
#> [1] 13.6
#>
#> $harvest_t
#> [1] 10.61
#>
#> $best_farmer
#> [1] "F034"
```

```
r$harvest_t
```

```
#> [1] 10.61
```

Appliquer une fonction à chaque groupe

Vos fonctions marchent partout où marche une fonction intégrée : dans `tapply()`, `sapply()`, `aggregate()`, `summarise()`.

```
se <- function(x) sd(x, na.rm = TRUE) / sqrt(sum(!is.na(x)))
round(tapply(seaweed$sgr, seaweed$technique, se), 4)
```

```
#>  long-line off-bottom
#>     0.0193     0.0175
```

```
# sapply over a vector of villages: one call of farm_report() each
sapply(c("Lovobe", "Mahavelo", "Befasy"),
       function(v) farm_report(seaweed[seaweed$village == v, ])$harvest_t)
```

```
#>   Lovobe Mahavelo   Befasy
#>    10.61    19.81     7.38
```

```
library(dplyr)
seaweed %>% group_by(zone) %>% summarise(mean = mean(sgr, na.rm = TRUE), se = se(s
gr))
```

```
#> # A tibble: 3 × 3
#>   zone    mean     se
#>   <chr>  <dbl>  <dbl>
#> 1 Centre  3.39 0.0255
#> 2 North   3.38 0.0230
#> 3 South   3.55 0.0231
```

`function(v) ...` écrite directement dans `sapply()` est une **fonction anonyme** : une petite fonction utilisée une fois. Depuis R 4.1 on peut l'écrire `\(v) ...` .

Ranger ses fonctions dans un fichier

Mettez vos fonctions dans `scripts/functions.R` (avec des commentaires expliquant chaque argument), et commencez chaque script d'analyse par `source("scripts/functions.R")` . Vous construisez votre propre petite boîte à outils, réutilisée d'un projet à l'autre.

```r
# scripts/functions.R ------------------------------------------------------------
# sgr(): specific growth rate in % per day
#   w_initial, w_final: weights (same unit); days: duration
sgr <- function(w_initial, w_final, days) 100 * log(w_final / w_initial) / days

# se(): standard error of the mean, ignoring NA
se <- function(x) sd(x, na.rm = TRUE) / sqrt(sum(!is.na(x)))
```

Travail d'application et quiz

Travail d'application I6

Script `scripts/i06_functions.R` et un fichier `scripts/functions.R` .

1. Écrivez `sgr(w_initial, w_final, days)` avec les vérifications ci-dessus. Combien vaut `sgr(8, 21, 42)` (3 décimales) ?
2. Écrivez `survival(stocked, alive)` qui renvoie le % d'animaux encore vivants. Avec `seacucumber_pens.csv` et `seacucumber_harvest.csv` (jointure par `pen_id`), calculez la survie de chaque enclos. Survie moyenne (2 décimales) ? Quelle ferme a la survie moyenne la plus élevée ?
3. Écrivez `se(x)` . Erreur standard du SGR des lignes **off-bottom** (4 décimales) ?
4. Écrivez `harvest_t(d)` qui renvoie la récolte totale en tonnes d'un tableau `d` . Valeur pour la zone **Centre** (3 décimales) ?
5. Écrivez `cv(x)` , le coefficient de variation en % (`100 * sd / mean` , sans NA). CV de toutes les valeurs de SGR (2 décimales) ? Avec `tapply()` , quel village a la croissance la plus variable (CV le plus élevé) ?

Question 2 : `ph <- merge(pens, harvest, by = "pen_id")` , puis `ph$survival <- survival(ph$juveniles_stocked, ph$alive_at_harvest)` .

Quiz : répondez en ligne sur le site du cours (votre score y est enregistré).

by Aroo Falinirina